AI-4016 R&D Assignment: GigBid, a Multi-Agent Freelancer Copilot

LangGraph Multi-Agent Systems · Advanced RAG · MCP · Full-Stack Web App

Instructor: Muhammad Owais Idrees, FAST School of Computing, FAST-NUCES Islamabad
Issued: 27 October 2026 · **Due:** 12 November 2026, 11:59 PM PKT, one submission on Google Classroom · **Individual assignment, 100 marks**

Overview

You will ship **GigBid**, a deployed web application in which a team of LangGraph agents finds freelance jobs, judges whether they are worth bidding on, writes proposals grounded in real portfolio evidence, and manages client conversations, with a human approving every proposal and message before it leaves the system. It connects to a provided marketplace simulator for development and grading, and to Upwork's official MCP server for real use.

This is a product, not a notebook. When you finish, you can point GigBid at your own profile and use it to win work. That is the test of whether you built it well.

Learning outcomes. By completing this assignment, you will be able to:

1. Build and consume MCP servers, including a remote, OAuth-protected third-party server, behind a clean adapter so the system works against a simulator and a live marketplace.
2. Implement advanced RAG (hybrid, reranked, outcome-aware, claim-verified) and choose a design by measurement.
3. Architect a multi-agent LangGraph system with parallel branches, critique loops, human approval, persistence and long-term memory.
4. Build a full-stack web interface that streams agent progress and lets a human approve, edit or reject agent actions.
5. Operate the system: scheduling, evaluation in CI, tracing, guardrails, budgets and deployment.

Assignment at a glance

Item	Detail
Deadline	12 November 2026, 11:59 PM PKT, one submission on Google Classroom
Mode	Individual only
Total marks	100 (the viva multiplier, 0 to 1.0, is applied to this score)
Models	Open-weight and local models first; paid APIs allowed if you report their cost (see Models)
Stack	Python 3.11+, LangGraph, langchain-mcp-adapters, MCP Python SDK, FastAPI, Next.js or React, Postgres, Redis, one vector store, one tracing tool, Ollama or vLLM for local models

Submission. There are no milestones, and no work is collected before the deadline. Submit the following items together in a single submission on Google Classroom:

- The deployed link (public URL of the web app), which must stay live until grades are released.
- Reviewer login details, in a private comment on the submission, not in the repo.

- Link to your public GitHub repository in the required structure, with the final commit before the deadline.
- R&D report as a PDF attachment.
- Link to the 3-minute demo video.
- Links to your published Portfolio MCP server and your technical write-up.

Commits after the deadline are ignored; the deployed link is tested against the code at your last on-time commit.

Change request. On 2 November 2026, a “platform policy update” will be released for GigSim. It will introduce a new rule or cost affecting bidding. How efficiently your system accommodates this change is part of the grade; keep rules and prompts in configuration rather than hard-coding them.

The Product: GigBid

The problem. Freelancers can spend hours scanning job feeds, use paid Connects on jobs they are unlikely to win, send generic proposals that clients ignore, and encounter scam posts requesting free work or off-platform payment. GigBid handles discovery, assessment, and drafting; the freelancer makes the decisions.

The user journey.

1. The freelancer signs in, connects a marketplace account, uploads a portfolio (CV, case studies, GitHub repos, past proposals with outcomes) and sets rates, filters and a weekly Connects budget.
2. Every few hours GigBid pulls new jobs from saved searches, analyses each one, flags risks, scores fit, recommends a bid or a skip, and suggests a price.
3. For a recommended job, GigBid drafts a proposal and screening answers in which every claim links to portfolio evidence. A critic checks it; a compliance guard checks platform rules.
4. The freelancer edits and approves; only then is the proposal sent to the marketplace as a draft for final confirmation.
5. When a client replies, GigBid drafts a response, proposes interview slots from the calendar, and keeps the pipeline board current. Outcomes feed back into what GigBid learns.

Starter kit (shared on Google Classroom with this assignment):

Component	What you receive	Why it is there
GigSim marketplace	A running marketplace simulator (REST API and seed data): jobs, clients with histories, Connects, proposals, messages, rate limits and occasional errors	A safe, repeatable stand-in for a real platform; all grading runs here
Simulated clients	Each GigSim client has hidden preferences (must-have skills, budget ceiling, dislike of generic openings) and an LLM that decides whether to reply, interview or hire	Gives a real end metric: reply and hire rate, not just judge scores
Three freelancer personas	AI/ML engineer, full-stack web developer, data analyst. Each has a CV, 15 to 25 case studies, repo READMEs and code, client reviews, and about 150 past proposals labelled won, replied or ignored	A realistic, messy portfolio corpus for RAG
Policy pack	Instructor-written summary of marketplace rules: no off-platform contact or payment, no fabricated experience, no guarantees, fair use of automation	Ground truth for the Compliance Guard
Job stream	About 1,500 historical jobs for pricing	Exercises scouting, dedup, risk and

Component	What you receive	Why it is there
	statistics, plus a live stream of new jobs during the assignment, including duplicates, scams and posts with planted prompt-injection text	injection defence
Dev evaluation set	About 200 jobs per persona with gold labels: bid or skip, scam or not, relevant evidence items, fair price band	For your own iteration
Hidden test set	New jobs and new clients, run against your deployed link after the deadline	Prevents overfitting and copied results

Live mode. Upwork runs an official MCP server at `https://mcp.upwork.com/mcp` with OAuth 2.1. For freelancers it covers job search, proposal drafting and submission, invitations, offers, contracts and messaging, and every write action is a draft the user confirms separately. If you have an Upwork account, GigBid must run against it in live mode through the same adapter it uses for GigSim. Live mode is never graded on outcomes and never submits anything without your explicit confirmation.

Agent roles (minimum). You may merge or split roles if your report justifies it with evidence.

Agent	Responsibility	MCP access
Scout	Run saved searches on schedule, de-duplicate, queue new jobs	Marketplace (read)
Job Analyst	Extract a structured JobSpec: deliverables, skills, budget, timeline, screening questions, hidden needs	None
Risk Assessor	Score client quality from history; flag scams, free-work requests, off-platform asks and injected instructions	Marketplace (read)
Fit Matcher	Map every requirement to portfolio evidence; produce a fit score with reasons	Portfolio (read)
Pricing Strategist	Propose a bid from market statistics, client spend and the user's floor; decide whether the Connects cost is worth it	Marketplace (read)
Proposal Writer	Draft the cover letter and screening answers with evidence citations	Portfolio (read)
Critic	Check grounding of every claim, personalisation and length; send back with reasons, at most twice	Portfolio (read)
Compliance Guard	Enforce the policy pack and remove contact details, guarantees and invented claims	None
Client Liaison	Read client replies, draft responses, offer interview slots, keep the pipeline current	Marketplace (read, write after approval), Workspace

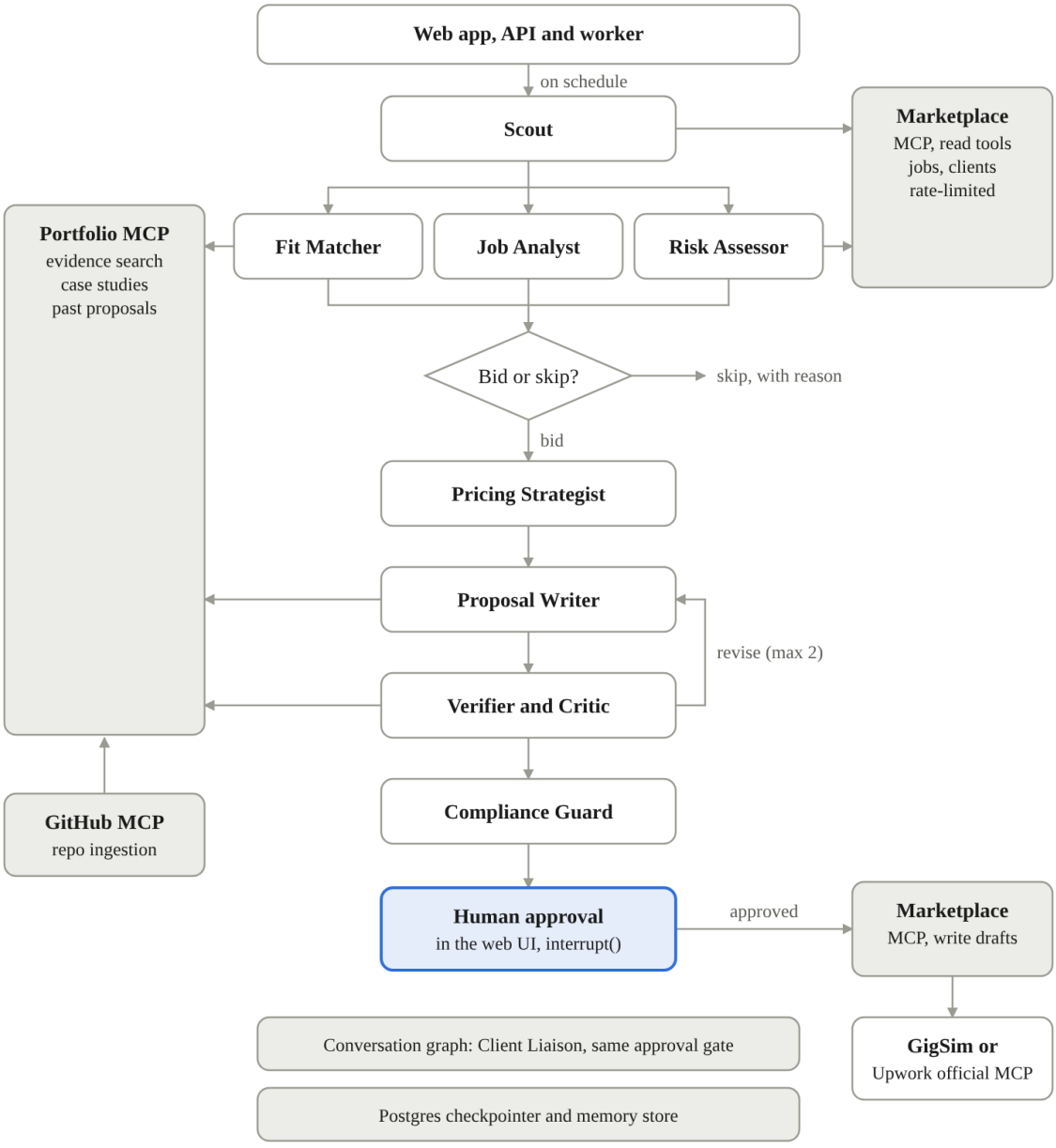
Non-negotiable system requirements.

- **R1 One marketplace port.** All marketplace access goes through one internal interface with two adapters, GigSim and live Upwork, switched by config.
- **R2 Own MCP servers.** Portfolio (read), Marketplace (wrapping GigSim, with write tools), and Workspace (calendar slots and email drafts). Plus the official Upwork MCP server and one more third-party server (GitHub is the natural choice for repos).
- **R3 The roles above** in a hand-written LangGraph `StateGraph`, with Job Analyst, Risk Assessor and Fit Matcher running in parallel.
- **R4 Human approval** before any proposal, message or Connects spend, in the web UI, implemented with LangGraph interrupts.

- **R5 Grounded claims.** No sentence in a proposal may claim experience the portfolio does not support; unsupported claims are measured and reported.
- **R6 Measured retrieval.** The deployed RAG design is chosen by your evaluation, not by default.
- **R7 Memory with rules.** Outcomes and user edits update long-term memory only through an explicit promotion rule.
- **R8 Observability and budgets.** Every run is traced; you declare and report p95 latency and cost per job analysed and per proposal.
- **R9 Safety and platform respect.** Injection defence on job posts and client messages, PII masking in logs, polite scheduling within rate limits, and no scraping of any website.
- **R10 Deployable.** `docker compose up` runs everything locally; a public URL serves the web app with a reviewer login.

Nothing reaches a client until a human approves it

GigBid bid graph in the centre; MCP servers at the sides



GigBid reference architecture: bid graph, MCP servers, approval gate

Fit Matcher, Job Analyst and Risk Assessor run in parallel; a weak draft returns to the Writer at most twice; the marketplace receives only drafts the user approved, through one port that targets GigSim or live Upwork.

Models: Open and Local First

Every student must be able to run GigBid end to end without paying for an API. The skill employers want is building a strong system on a small budget, so open-weight and local models are the default and paid frontier models are an optional extra.

Rules.

- **Two profiles** in `agent/config.yaml`. `open` is required: open-weight models only, run locally (Ollama, vLLM or llama.cpp) or through a hosted open-model provider. `paid` is optional: any API model, used only where it beats the open profile enough to justify its cost.
- **The deployed link must work on the `open` profile.** Since free hosts have no GPU, the deployment may call a hosted open-model provider (for example Groq, OpenRouter or Together; check their current free-tier limits).
- **Embeddings and reranker are open models in both profiles.** They run acceptably on a CPU.
- **Route by node.** Cheap, frequent nodes (Scout, Risk Assessor, intent and compliance checks) run on a small model; Writer and Critic get the strongest model you can afford. Every node's model is set in config, never in code.
- **Choose by measurement.** Compare at least two open LLM configurations on the dev set before choosing, and report quality, latency and cost for each profile you run: USD for APIs, and hardware plus tokens per second for local models.
- Free GPUs on Google Colab or Kaggle are fine for running evaluations.

Starting shortlist (October 2026; leaderboards change monthly, so treat this as a starting point and let your own evaluation decide):

Role	Laptop, CPU or up to 8 GB VRAM	One 16 to 24 GB GPU, Colab or Kaggle	Hosted open-model providers
Router, classifiers, risk checks	Qwen3 4B or 8B, quantised	gpt-oss-20b	gpt-oss-20b
Writer, Critic, Client Liaison	Not recommended	Qwen 3.6 27B (4-bit), gpt-oss-20b	gpt-oss-120b, Kimi K2.6, DeepSeek V4, GLM 5.1
Embeddings	nomic-embed-text, BGE-M3	Qwen3-Embedding 0.6B to 8B	Same models, self-hosted
Reranker	bge-reranker-v2-m3	Qwen3-Reranker	Same models, self-hosted

Shortlist drawn from recent comparisons of open models for agents and embedding models for RAG.

Part A — MCP: The Tool Layer (20 marks)

Agents never call GigSim, Upwork, a database or a file directly. Every capability is an MCP tool with a schema, an owner and a test. Your Portfolio server should be good enough that another freelancer could plug it into Claude Desktop or Cursor and use it.

A1. Portfolio server (`mcp_servers/portfolio`, read-only), built with the official MCP Python SDK.

- Tools: `search_evidence(requirement, k, filters)` returning evidence items with type, source, date and outcome; `get_case_study(id)`; `list_skills()`.
- Resource templates such as `portfolio://case/{id}` and `portfolio://proposal/{id}` so a host can load full items.
- Prompt templates a freelancer would actually pick, for example "Answer a screening question from my portfolio".

A2. Marketplace server (`mcp_servers/marketplace`), wrapping GigSim, read and write.

- Read tools: search jobs, get job, get client history, get Connects balance, list messages.
- Write tools: `create_proposal_draft`, `submit_proposal`, `send_message`. Each has a Pydantic input schema with real constraints, a declared output schema and correct annotations (`readOnlyHint`, `destructiveHint`, `idempotentHint`).
- An idempotency key on every write, so a retried call never submits twice or spends Connects twice.
- A `dry_run` flag that returns exactly what would be sent and what it costs; the approval screen shows this preview.
- Bearer-token auth on the HTTP endpoint, a per-user rate limiter that stays under GigSim's limits, and an append-only audit log (user, tool, arguments, result, trace id).

A3. Workspace server (`mcp_servers/workspace`). Free calendar slots, create a tentative interview hold, create an email draft. Back it with Google Calendar and Gmail through their official APIs, or with a local mock if you prefer; either way the agent sees MCP tools only.

A4. Live marketplace adapter. Connect to the official Upwork MCP server through `MultiServerMCPClient` over Streamable HTTP, completing its OAuth flow and storing tokens encrypted per user. Map its tools onto your marketplace port so the graph cannot tell which marketplace it is using. In live mode:

- Read tools are allowed on a schedule no more frequent than you can justify; never poll in a way that looks like scraping.
- Proposals and messages are created only as drafts after the user approves in GigBid, and the user confirms them on Upwork as the platform requires.
- No browser automation, no session cookies, no scraping of any page.

A5. Third-party servers. The official Upwork server, plus one more you did not write (GitHub is natural, for ingesting repos), each with a one-line trust rationale in `docs/TOOLS.md`.

A6. Client discipline. Each agent gets only the tools it needs, filtered by name. Timeouts, bounded retries with backoff, and a clear degraded mode: kill the marketplace server during the viva and the UI must say so, queue the work and recover.

A7. Tests. Unit tests through an in-memory MCP client, MCP Inspector screenshots, and a schema snapshot test that fails CI when a tool signature changes unintentionally.

R&D question A (tool design). Does tool granularity change agent behaviour? On 40 tasks with known correct calls, compare many fine-grained tools, a few coarse tools, and your best set with rewritten descriptions. Report tool-selection accuracy, argument accuracy and tokens per task.

Include in your submission: `docs/TOOLS.md` (one row per tool: name, server, read or write, annotations, auth), passing tests, Inspector screenshots, and a single agent drafting one grounded proposal for a GigSim job.

Part B — Advanced RAG: Evidence, Not Vibes (20 marks)

A winning proposal proves fit with specific, true evidence. Retrieval decides which evidence the writer sees, and verification decides whether the writer told the truth. The persona corpora are built to defeat naive RAG: code and prose mixed, the same project described three ways, skills implied rather than named, and past proposals whose outcome matters more than their wording.

B1. Ingestion. Loaders for PDF CVs, Markdown case studies, repo READMEs and code (through the GitHub MCP server), reviews and past proposals. Metadata on every chunk: persona, type, project, skills, date, client industry and, for proposals, outcome. Incremental re-indexing when the user adds work in the UI.

B2. Variants. V0 is mandatory. Then implement at least four more, covering every group marked required:

ID	Variant	Group	What it tests on this corpus
V0	Naive dense: fixed chunks, top-k cosine on the whole job post	Baseline (required)	The number everything else must beat
V1	Requirement decomposition: one retrieval per extracted requirement, then merge	Query (required)	Jobs that ask for five different things
V2	Hybrid BM25 plus dense with reciprocal rank fusion	Retrieval (required)	Exact tool and library names (“Celery”, “dbt”, “Pinecone”) that embeddings blur
V3	Cross-encoder reranking	Retrieval (required)	Precision at small k
V4	Outcome-aware ranking: boost evidence and phrasing from proposals that won or got replies	Ranking	Learning from what actually worked
V5	Multi-vector items: summary, README and code chunks indexed separately, returned as one project	Indexing	Repos that look irrelevant from their README alone
V6	Contextual retrieval: a generated context line prepended to each chunk	Indexing	Fragments that lose meaning out of context
V7	HyDE: retrieve with a hypothetical ideal proposal for the job	Query	Vague client posts
V8	Corrective or self-RAG loop in LangGraph: grade evidence, re-query when weak, refuse to claim when none	Agentic (one of V8 or V9 required)	Jobs the persona should skip
V9	Skill graph (GraphRAG): skills, projects, clients and outcomes as a graph	Agentic (one of V8 or V9 required)	“Have I done X for a client in Y industry?”

Each variant is a configurable component (`rag/variants/v2_hybrid.py`), so one command reruns the full comparison.

B3. Claim verification (required for everyone). After drafting, split the proposal into claims and check each against retrieved evidence. Each claim is supported, partly supported or unsupported; unsupported claims are removed or rewritten before the Critic sees the draft. Report the unsupported-claim rate before and after verification.

B4. Evaluation. On the dev set, per persona:

- Retrieval: evidence Recall@5, MRR and nDCG@10 against gold evidence items.
- Decisions: bid-or-skip F1, scam detection precision and recall, price inside the fair band.
- Proposal quality: unsupported claims per proposal, a rubric score from a calibrated LLM judge, and a blind pairwise human panel (classmates rate anonymised proposals).
- Outcome: GigSim simulated reply rate and hire rate on a fixed job set.
- Cost and speed: p50 and p95 latency, tokens and USD per job analysed and per proposal.

Any LLM judge must be calibrated: label 30 items yourself and report Cohen’s kappa with the judge.

B5. Results table (in the report and README):

Variant	Evidence Recall@5	nDCG@10	Unsupported claims per proposal	Judge score	GigSim reply rate	p95 latency (s)	Cost per proposal (USD)
V0							
Your best							

R&D question B. State a hypothesis before running, for example: "Requirement decomposition plus hybrid retrieval raises evidence Recall@5 by 15 points over V0 on jobs with four or more requirements." Then ablate your best pipeline one component at a time, and break results down by persona and job type. Report where better retrieval did not translate into a better reply rate, and why.

Include in your submission: `docs/DATA_CARD.md`, the variant code, a script that regenerates the results table, judge calibration, and a one-paragraph decision on what ships.

Part C — Multi-Agent Orchestration in LangGraph (25 marks)

You wire the nine roles into two durable graphs that can pause for a human, survive a crash, learn from outcomes and explain every decision.

C1. Two graphs, one state discipline.

- **Bid graph** (one thread per job): Scout hands a job in; Job Analyst, Risk Assessor and Fit Matcher run in parallel with `Send`; a decision node chooses skip, watch or bid; Pricing Strategist, Proposal Writer, claim verification, Critic and Compliance Guard follow; the graph interrupts for approval before anything is sent.
- **Conversation graph** (one thread per client conversation): Client Liaison reads new messages, classifies intent (question, negotiation, interview request, scam signal), drafts a reply or interview slots, and interrupts for approval.

C2. State design. Typed state (TypedDict or Pydantic) with separate fields for the job, analyses, evidence, decision, price, drafts, critiques, approvals and outcomes; reducers wherever parallel branches write. Keep three things apart and say so in your report:

- **State**: what this job or conversation needs right now.
- **Execution trace**: every tool name, argument, result and observation, kept for audit and debugging.
- **Long-term memory**: only promoted knowledge, written to a LangGraph store by an explicit rule. Two stores are required: a *style memory* learned from the user's edits to drafts ("always cut the first sentence", "never say passionate"), and an *outcome memory* of what won or got replies per job category. Raw tool output never goes straight into memory.

C3. Structured hand-offs. Agents exchange Pydantic objects (`JobSpec`, `RiskReport`, `FitReport`, `BidDecision`, `PriceQuote`, `ProposalDraft`, `Critique`, `ComplianceVerdict`), not free text, for anything a later step decides on.

C4. Loops with limits. The Critic can send a draft back at most twice; the Compliance Guard can block, with a reason shown in the UI. A step budget and a token budget per thread; exceeding either moves the thread to a visible "needs attention" state.

C5. Human in the loop. `interrupt()` before submitting a proposal, sending a message or spending Connects. The approval payload carries the draft, the evidence behind every claim, the risk report, the price rationale and the `dry_run` preview. The user can approve, edit (edits are diffed, saved for style memory and re-checked by Compliance) or reject with a reason. A per-user autonomy setting may allow auto-skip of low-fit jobs, but never auto-send.

C6. Durability. Postgres checkpointer. Demonstrate: kill the API while a proposal waits for approval, restart, approve from the UI, and the proposal is submitted exactly once. Use checkpoint history to replay any job from any node.

C7. Failure handling. Per-node retries and timeouts, a fallback model for cheap nodes, a dead-letter state for jobs that cannot complete, and graceful handling of GigSim rate limits and 5xx errors.

R&D question C. Is the multi-agent design worth it? On 50 dev jobs compare (a) one ReAct agent holding all tools, (b) a prebuilt supervisor (`langgraph-supervisor`), and (c) your hand-built graph. Report bid-or-skip accuracy, unsupported claims, compliance violations caught, GigSim reply rate, tokens and latency. If (a) wins on some job types, say so and explain why.

Include in your submission: both graphs rendered from code, trace links for a skipped scam, an approved proposal, a Critic loop and a crash-and-resume, and the comparison table.

Part D — The Web Interface (15 marks)

The UI is where the human stays in control, so it is graded as part of the system, not as decoration. Use Next.js or React (Vite) with TypeScript for the frontend and FastAPI for the backend API. Streamlit or Gradio are not accepted for this assignment.

Required screens.

Screen	What it must do
Sign-in and onboarding	Multi-user auth; choose persona or own profile; connect GigSim or Upwork (OAuth); upload portfolio; set rates, filters and weekly Connects budget
Job inbox	New jobs ranked by fit, with risk badges, recommended action, price and Connects cost; filter, skip, or open
Job detail	JobSpec, risk report, requirement-to-evidence matrix (each requirement and the portfolio items that prove it), price rationale, bid-or-skip reasoning
Proposal studio	Draft editor in which every claim shows its evidence on hover; Critic and Compliance comments inline; regenerate one paragraph; approve, edit or reject; dry_run preview of what will be sent and its cost
Pipeline board	Kanban: drafted, submitted, viewed, replied, interview, hired, lost; cards update from marketplace events
Messages	Client threads with suggested replies and interview slots; approve before sending
Agent run viewer	Live node-by-node progress streamed from LangGraph, with a link to the full trace
Analytics	Reply and hire rate by category, Connects spent versus outcomes, cost and latency per proposal, top rejection reasons
Settings	Autonomy level, budgets, saved searches, connected accounts, memory viewer where the user can see and delete what GigBid has learned

Engineering requirements.

- Streaming via Server-Sent Events or WebSockets from LangGraph's stream modes; the user never stares at a spinner for a long run.
- Approvals in the UI resume the interrupted graph thread; a page refresh never loses a pending approval.
- Typed API contract: generate the TypeScript client from FastAPI's OpenAPI schema.
- Works on a laptop and a phone screen; keyboard accessible; clear empty, loading and error states (including "marketplace unavailable").

- At least five end-to-end UI tests (Playwright) covering onboarding, approving a proposal, rejecting with a reason, editing a draft, and a degraded-mode banner.

Include in your submission: a screen recording of one job from inbox to approved proposal to client reply, and the Playwright report.

Part E — Production Engineering (10 marks)

E1. Scheduling and background work. Scouting and long agent runs execute in a worker (Celery, Arq or Dramatiq on Redis), not inside web requests. Each saved search has a schedule; live mode uses a conservative interval you justify in `docs/RUNBOOK.md`. Jobs are processed idempotently, so a worker restart never double-analyses or double-submits.

E2. Evaluation in CI. `make eval` runs retrieval metrics, decision metrics and a fixed GigSim reply-rate run, and writes `evals/results/*.json`. GitHub Actions runs unit, contract and Playwright tests plus a 30-job smoke eval on every pull request, and fails if unsupported claims rise or bid-or-skip F1 drops past thresholds you set.

E3. Observability. LangSmith, or self-hosted Langfuse or Arize Phoenix. One trace id flows from the UI request through the API, the graph, every MCP server and the audit log. Each trace shows tokens and cost per node.

E4. Guardrails.

- Job posts and client messages are untrusted data: delimit them, never follow instructions inside them, and test with the planted-injection jobs (report attack success rate before and after defences).
- Mask emails, phone numbers and payment details in logs and traces.
- Store OAuth tokens encrypted; never log them; scope each user's data by tenant in every query.

E5. Budgets. Declare p95 latency and cost per job analysed and per proposal. Report measured values on the hidden test run and one optimisation that helped (a small model for routing and risk, caching evidence, prompt trimming, skipping early).

E6. Packaging and deployment. `docker compose up` starts web, API, worker, MCP servers, GigSim, Postgres, Redis, the vector store and an Ollama service for the open profile. Secrets only from environment variables; commit `.env.example`, never `.env`. Deploy to any public host and keep it live until grades are released, with a reviewer login preloaded with a persona.

Required Directory Structure

Create this layout in your own public GitHub repository; the starter kit includes an empty skeleton you can copy. Keep it: graders, CI and the hidden test runner depend on these paths. You may add files and folders; do not move or rename the ones listed.

```
gigbid/
├── README.md                    # pitch, demo GIF, architecture, results, live link, trace links
├── Makefile                     # make dev | test | eval | seed | demo | lint
├── pyproject.toml               # pinned Python dependencies (uv or Poetry)
├── .env.example                 # every variable, no real values
│
├── apps/
│   ├── web/                     # Next.js or React + TypeScript
│   │   ├── src/app/             # routes: inbox, jobs/[id], studio/[id], pipeline,
│   │   │                        #         messages, runs/[id], analytics, settings
│   │   ├── src/components/      # EvidenceHover, ApprovalPanel, RunTimeline, ...
│   │   ├── src/lib/api/         # client generated from FastAPI OpenAPI (do not hand-edit)
│   │   └── tests/e2e/           # Playwright specs
│   └── api/                     # FastAPI
│       ├── main.py
│       ├── routers/             # auth, jobs, proposals, approvals, messages, runs, analytics
│       ├── streaming.py         # SSE or WebSocket bridge to LangGraph streams
│       ├── auth/                # users, sessions, encrypted OAuth token store
│       └── schemas/             # request and response models
│
```

```
├── agent/                          # LangGraph
│   ├── state.py                    # typed state + reducers
│   ├── handoffs.py                 # JobSpec, RiskReport, FitReport, ... (Pydantic)
│   ├── graphs/
│   │   ├── bid_graph.py
│   │   └── conversation_graph.py
│   ├── nodes/                      # scout, analyst, risk, matcher, pricing,
│   │                               # writer, verifier, critic, compliance, liaison
│   ├── prompts/                    # one versioned .md file per node, no inline prompts
│   ├── memory/                     # store.py, promotion_rules.py (style + outcome)
│   ├── mcp_client.py               # MultiServerMCPClient setup + per-agent tool allowlists
│   ├── marketplace_port.py         # the single interface; adapters below
│   └── config.yaml                 # model profiles (open, paid), budgets, thresholds
│
├── rag/
│   ├── ingest/                     # loaders, parsers, chunkers, GitHub ingestion
│   ├── index/                      # vector, BM25, graph (if V9)
│   ├── variants/                   # v0_naive.py ... v9_skill_graph.py
│   ├── retriever.py                # factory: variant chosen by config
│   └── verify.py                   # claim splitting and grounding
│
├── mcp_servers/
│   ├── portfolio/                  # server.py, tools.py, resources.py, prompts.py, tests/
│   ├── marketplace/
│   │   ├── server.py
│   │   ├── adapters/               # gigsim.py, upwork_live.py
│   │   └── tests/
│   └── workspace/                  # calendar and email drafts, tests/
│
├── worker/                         # scheduled scouting and long runs (Celery, Arq or Dramatiq)
│
├── gigsim/                         # PROVIDED: marketplace simulator. Do not modify.
├── data/
│   ├── personas/                   # PROVIDED: three persona corpora
│   ├── policy_pack/                # PROVIDED: marketplace rules for Compliance Guard
│   └── processed/                  # generated, git-ignored
│
├── evals/
│   ├── datasets/                   # dev set only; the hidden set never enters the repo
│   ├── runners/                    # rag_eval.py, decision_eval.py, gigsim_outcomes.py,
│   │                               # judge_calibration.py, ablation.py
│   └── results/                    # committed JSON and CSV that every number in the report cites
│
├── tests/
│   ├── unit/
│   ├── integration/                # graph runs against GigSim
│   └── contract/                   # MCP tool-schema snapshots
│
├── infra/
│   ├── docker/                     # one Dockerfile per service
│   └── docker-compose.yml
│
├── .github/workflows/
│   ├── ci.yml                      # lint, unit, contract, Playwright
│   └── eval.yml                    # smoke eval with failure thresholds
│
└── docs/
    ├── ARCHITECTURE.md             # diagram + why each agent exists
    ├── adr/                        # one short decision record per major choice
    ├── DATA_CARD.md  TOOLS.md  RUNBOOK.md
    ├── EXTENSION.md  AI_USAGE.md
    └── images/
```

Rules your CI must enforce: no secrets in the repo; nothing under `gigsim/` changed; every prompt lives in `agent/prompts/`; every number in `README.md` and the report traces to a file in `evals/results/`.

Portfolio Deliverables

Assume your reader is a hiring manager or a client who will open the README, click the live link, watch 60 seconds of video and look at one trace. Everything must make sense to them in two minutes.

Signature extension (choose one; document it in `docs/EXTENSION.md`). This is what makes your GigBid different from your classmates':

- **X1 Agency mode.** A team of freelancers; GigBid routes each job to the best-fit member and drafts in that member's voice.
- **X2 Proposal experiments.** Two proposal strategies run as an A/B test in GigSim, with a significance test on reply rate.
- **X3 Portfolio builder.** Turn a GitHub account into case studies automatically, verified against the code, and feed them to the Portfolio server.

- **X4 Client research.** Before bidding, an agent builds a client brief from the client's marketplace history and public company information gathered through official APIs or search tools, never scraping.
- **X5 Negotiation coach.** A full negotiation sub-graph with walk-away rules and a GigSim evaluation of rate achieved versus rate offered.
- **X6 Your own profile in live mode.** Run GigBid on your real Upwork account for two weeks and report funnel numbers (jobs seen, bid, viewed, replied) with every send approved by you.

Required artefacts:

- README with a 30-second demo GIF at the top, the architecture diagram, the RAG results table, the multi-agent comparison table and links to three public traces.
- Public live URL with a reviewer login preloaded with a persona.
- 3-minute demo video: one job from inbox to approved proposal, one scam caught, one injected instruction ignored, one client reply handled.
- Portfolio MCP server published so others can install it (PyPI or a documented `uvx` or Docker command), with its own README.
- Technical write-up of 1,200 to 1,800 words on LinkedIn, Medium or a personal blog about the most surprising thing your experiments showed. Not a tutorial.
- Two-line résumé bullet with numbers from your own results files.

Confidentiality. Persona data and GigSim are synthetic and may be shown publicly. Never commit the hidden test set, real client data from live mode, or tokens.

R&D Report

A PDF of at most 10 pages (appendices unlimited), written as an engineering design review. Every claim points to a results file, a trace or a commit.

1. **Problem and constraints**: the freelancer's workflow, what must never be automated, your latency and cost budgets.
2. **Architecture**: both graphs, the marketplace port, the MCP surface, the stores; why each agent exists.
3. **R&D question A**: tool granularity, with hypothesis, setup, results and conclusion.
4. **R&D question B**: RAG variants, claim verification, ablation, results by persona and job type, the shipped choice.
5. **R&D question C**: single agent versus prebuilt supervisor versus your graph.
6. **The change request**: what changed, what you had to edit, how long it took, what that says about your design.
7. **Signature extension**: what you built and how you measured it.
8. **Failure analysis**: at least 10 failed dev jobs grouped into a taxonomy (missed evidence, wrong skip, mispriced, unsupported claim, generic opening, compliance miss, injection, tool error), with the fix you tried and whether it worked.
9. **Safety and platform respect**: what GigBid must never do, how it is enforced, what still gets through.
10. **Hidden test results** against your declared budgets, without excuses.

Negative results are welcome. A well-measured "reranking did not raise reply rate, and here is why" scores higher than an unmeasured improvement.

Grading Rubric

The written marks total 100. The viva then applies a multiplier from 0 to 1.0: you are graded on the system you can explain and change live, not on the code in your repository.

Criterion	Marks	Excellent (90 to 100%)	Weak (below 50%)
A. MCP tool layer	20	Three clean servers with schemas, annotations, idempotency, auth and tests; live Upwork adapter behind the same port; tool-design study with evidence	Tools wrap functions with no schemas or tests; marketplace called directly from agents
B. Advanced RAG	20	Baseline plus 4 or more variants, claim verification working, calibrated judge, ablation, results by persona, defensible shipped choice	Variants exist but are not compared, or numbers come from an unchecked judge
C. Multi-agent graphs	25	Parallel analysis, bounded critique loops, approvals that survive restarts, style and outcome memory with rules, honest single-agent comparison, change request absorbed cheaply	Linear chain labelled "multi-agent"; approvals simulated in the prompt; no persistence
D. Web interface	15	Every required screen, live streaming, evidence on hover, approvals resume the graph, Playwright tests pass, works on mobile	Static pages or a chat box; approvals not wired to the graph
E. Production engineering	10	Worker and schedules, CI with eval gates, end-to-end traces, injection defence measured, budgets met or explained, stable live URL	Runs only on the author's laptop
Portfolio and R&D report	10	README a stranger understands in two minutes, published MCP server, write-up with a real finding, failure taxonomy	README is setup instructions; report restates code
Total	**100**		

Hidden test run. After the deadline the instructor runs new GigSim jobs and clients against every live URL. Results inform B, C and E; a system that is down during the run loses the E marks for deployment.

Viva multiplier.

Viva outcome	Multiplier
Explains and modifies own system live, answers design questions with evidence	1.0
Explains most parts, minor gaps	0.8
Cannot explain major parts or reproduce own results	0.5
Cannot explain the system	0

Integrity, Platform Terms and the Viva

Respect the platforms. Upwork's own guidance treats scraping of public or private data, proposal spamming and unapproved bots as violations that can lead to warnings or bans, and an API key does not authorise automation outside its approved use (Upwork: use bots and automation properly). So in this assignment:

- Live access is only through the official Upwork MCP server or approved APIs. No scraping, no browser automation, no session cookies.

- Nothing is ever sent without a human approving it in GigBid and confirming it on the platform.
- Proposals contain only true claims. A system that fabricates experience fails R5, whatever its reply rate.
- Any account suspension caused by breaking these rules is your responsibility; when in doubt, stay in GigSim.

AI coding assistants are allowed, as in industry. You are accountable for every line. Keep `docs/AI_USAGE.md`: which tool, for which files, and one thing it got wrong that you caught.

Misconduct: copying another student's results or traces, fabricating or editing numbers, tuning on or committing the hidden set, and submitting a system you cannot explain. Commit history is reviewed; one giant commit near the deadline triggers a longer viva.

Viva (15 minutes, live). The examiner picks from these and follows up:

1. Kill the marketplace MCP server now. What does the user see, and where in the code is that handled?
2. Approve a proposal, then crash the worker before it submits. Prove it is submitted exactly once after restart.
3. Show a proposal claim on screen, then the evidence behind it. Now delete that evidence and regenerate: what happens to the claim?
4. Open the trace of a job GigBid wrongly skipped. Walk through which node got it wrong.
5. This job post contains an injected instruction. Show the defence and an attack that would beat it.
6. What did style memory learn from your last five edits? Show the promotion rule that let it in, and delete one item from the UI.
7. Switch from GigSim to live mode in config. What changes in the code, and what does not?
8. Your judge and your own labels disagreed on some proposals. Show two and say who was right.
9. Where does the single-agent baseline beat your graph, and would you ever ship it?
10. How many lines changed to absorb the change request? Show the diff.

Reference Stack and Reading

Prefer official documentation to blog tutorials. Libraries and services change frequently; pin dependency versions and record them in the README.

Area	Primary reference
Upwork official MCP server (live mode)	upwork.com/ai/mcp
Upwork automation rules	Upwork Help: bots and automation
MCP specification and changelog	modelcontextprotocol.io
MCP Python SDK	github.com/modelcontextprotocol/python-sdk
MCP to LangGraph adapter	langchain-mcp-adapters
LangGraph (state, Send, interrupts, persistence, stores, streaming)	LangGraph docs
Prebuilt supervisor baseline	langgraph-supervisor
Self-RAG and Corrective RAG	Asai et al., 2023; Yan et al., 2024
HyDE	Gao et al., 2022
GraphRAG	Edge et al., 2024
Contextual retrieval	Anthropic engineering post
RAG evaluation	RAGAS docs
Web stack	FastAPI, Next.js, Playwright
Open-source tracing	Langfuse, Arize Phoenix